Adding Images to GitHub READMEs

https://gemini.google.com/app/0cad25d0bc03abe3

User prompt: how to add an image to a readme file in github

Response: Adding an image to a GitHub README file is straightforward using Markdown or HTML.

Here are the most common ways to do it:

Method 1: Using Markdown (Recommended)

If you have the image uploaded directly to your GitHub repository (for example, in an `assets` or `images` folder), you can use a standard relative link:

```
![Alt text for the image](path/to/image.png)
```

If the image is hosted online elsewhere (like a public URL or CDN), use the absolute URL:

```
![Alt text for the image](https://example.com/image.png)
```

Method 2: Drag and Drop (Easiest for GitHub.com UI)

If you are editing your `README.md` directly in the GitHub web browser interface:

1. Place your cursor where you want the image to go.
2. Drag and drop your image file from your computer directly into the text editor.
3. GitHub will automatically upload the image to an invisible repository folder and insert the correct Markdown code for you.

Method 3: Using HTML (For Resizing and Alignment)

Markdown doesn't natively support resizing or centering images. If you need more control over how the image appears, use an HTML `<img>` tag:

```
<p align="center">
  <img src="path/to/image.png" alt="Alt text" width="300" />
</p>
```

To make sure everything works correctly, verify that the image renders properly in the **Preview** tab before saving your changes.

What kind of image are you trying to add (e.g., a screenshot, a logo, a banner), and where is it currently saved?

User prompt: how do i choose a bool datatype, while making a database?

Response: How you choose a boolean (true/false) data type depends entirely on the **Database Management System (DBMS)** you are using, as each system handles booleans slightly differently.

Here is how the most popular databases handle boolean data types:

1. PostgreSQL

PostgreSQL has a dedicated native boolean type. It accepts `TRUE`, `FALSE`, or `NULL`.

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50),
    is_active BOOLEAN DEFAULT TRUE
);
```

2. MySQL / MariaDB

MySQL uses `BOOLEAN` or `BOOL` as synonyms for **`TINYINT(1)`**. Under the hood, it stores `1` for true and `0` for false.

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50),
    is_active BOOLEAN DEFAULT 1
);
```

3. SQLite

SQLite does not have a separate storage class for booleans. Instead, boolean values are stored as integers `0` **(false)** and `1` **(true)**. However, you can still declare the column as `BOOLEAN` for readability and type affinity.

```
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT,
```

```
    is_active BOOLEAN DEFAULT 1
);
```

4. Microsoft SQL Server (T-SQL)

SQL Server does not use the word `boolean`. Instead, it uses the **`BIT`** data type, which accepts `1`, `0`, or `NULL`.

```
CREATE TABLE users (
    id INT IDENTITY(1,1) PRIMARY KEY,
    username VARCHAR(50),
    is_active BIT DEFAULT 1
);
```

Which database management system (e.g., PostgreSQL, MySQL, SQLite) are you currently using for your project?

User prompt: what does this do? FOREIGN KEY (klienti_id) REFERENCES klienti(klienti_id), FOREIGN KEY (preces_id) REFERENCES preces(preces_id)

Response: This snippet of SQL code defines **foreign key constraints**, which are used to link two tables together and enforce a relationship between them (often called referential integrity).

Here is a breakdown of what each part does:

1. `FOREIGN KEY (klienti_id) REFERENCES klienti(klienti_id)`

- This line tells the database that the column named **`klienti_id`** in the current table is a foreign key pointing to the **`klienti`** table (likely a table storing customers or clients) and references its primary key column (`klienti_id`).
- **What it enforces:** The database will not allow you to insert a row into the current table unless the `klienti_id` you provide already exists in the `klienti` table.

2. `FOREIGN KEY (preces_id) REFERENCES preces(preces_id)`

- This line similarly states that the column **`preces_id`** in the current table refers to the **`preces`** table (likely meaning "products" or "items" in Latvian) and its primary key (`preces_id`).
- **What it enforces:** It ensures that every product reference in this table points to a valid, existing product in the `preces` table.

Why is this used?

You typically see this exact pattern in a **junction/join table** or an **order/transaction table** (for example, an order linking a specific customer (`klienti_id`) to a specific product (`preces_id`)).

It prevents "orphaned" data—meaning you can't accidentally assign an order to a customer or a product that doesn't exist in your database.

---

*User prompt: import sqlite3 #1. pieslēgties pie datu bāzes database = sqlite3.connect("hotel.db") cursor = database.cursor() #2.1. izveidot tabulu ar klientiem cursor.execute(""" CREATE TABLE IF NOT EXISTS guests(guest_id INTEGER PRIMARY KEY, guest_name TEXT NOT NULL, guest_surname TEXT NOT NULL) """) #2.2. izveidot tabulu ar precem cursor.execute(""" CREATE TABLE IF NOT EXISTS rooms(room_number INTEGER PRIMARY KEY, room_availability BOOLEAN DEFAULT 1) """) cursor.execute(""" CREATE TABLE IF NOT EXISTS staff(staff_id INTEGER PRIMARY KEY, staff_name TEXT NOT NULL, staff_surname TEXT NOT NULL, staff_job TEXT NOT NULL) """) #2.3. izveidot tabulu ar pasutijumiem cursor.execute(""" CREATE TABLE IF NOT EXISTS orders(order_id INTEGER PRIMARY KEY, guest_name TEXT NOT NULL, guest_surname TEXT NOT NULL, room_number INTEGER, order_finished BOOLEAN DEFAULT 1, staff_id INTEGER, FOREIGN KEY (guest_name) REFERENCES guests(guest_name), FOREIGN KEY (guest_surname) REFERENCES guests(guest_surname), FOREIGN KEY (room_number) REFERENCES rooms(room_number), FOREIGN KEY (staff_id) REFERENCES staff(staff_id),) """) #3. ievietot datus tabulās cursor.execute(""" INSERT OR IGNORE INTO guests(guest_id, guest_name, guest_surname) VALUES(1, 'Romans', 'Zubenko') """) cursor.execute(""" INSERT OR IGNORE INTO rooms(room_number, room_availability) VALUES(1, 1) """) cursor.execute(""" INSERT OR IGNORE INTO staff(staff_id, staff_name, staff_surname, staff_job) VALUES(1, "Antons", "Mahno", "Cleaner") """) cursor.execute(""" INSERT OR IGNORE INTO pasutijumi (order_id, guest_name, guest_surname, room_number, order_finished, staff_id) VALUES(1, 'Romans', 'Zubenko', 1, 1, 1) """) #4. savienot tabulas #5. saņemt datus cursor.execute(""" SELECT * FROM guests """) guests = cursor.fetchall() print("\nKLIENTI") for guests in guests: print(guests) cursor.execute(""" SELECT guests.guest_name, rooms.room_number, staff.staff_id FROM orders JOIN guests ON orders.guest_id = guests.guest_id JOIN rooms ON orders.room_number = rooms.room_number """) pasutijumi = cursor.fetchall() print("\nPASUTIJUMI") #saglabat database.commit() why doesn't this code work?*

Response: There are a few different bugs in your code causing it to fail. Here is a breakdown of what went wrong:

1. The Bugs Explained

- **Trailing Comma in Table Creation:** In your `orders` table definition, there is an extra comma right after `FOREIGN KEY (staff_id) REFERENCES staff(staff_id)**,** just before the closing parenthesis )`. SQLite throws a syntax error for trailing commas.
- **Table Name Mismatch on Insert:** You create a table named `orders`, but your `INSERT` statement tries to add data into a table named `pasutijumi` (`INSERT OR IGNORE INTO pasutijumi...`). SQLite will throw a `no such table` error.
- **Foreign Key and Column Mismatch:** Your `orders` table was created with `guest_name` and `guest_surname` columns, but your `SELECT` query attempts to join using `orders.guest_id = guests.guest_id`. Furthermore, foreign keys in SQLite must point to a `PRIMARY KEY`

or `UNIQUE` column (like `guest_id`), not text names.

- **Missing Join in Query:** Your query selects `staff.staff_id`, but never actually `JOIN`s the `staff` table.

Corrected Code

Here is the fully fixed and working version of your script:

```
import sqlite3

# 1. Pieslēgties pie datu bāzes
database = sqlite3.connect("hotel.db")
cursor = database.cursor()

# 2.1. Izveidot tabulu ar klientiem
cursor.execute("""
    CREATE TABLE IF NOT EXISTS guests(
        guest_id INTEGER PRIMARY KEY,
        guest_name TEXT NOT NULL,
        guest_surname TEXT NOT NULL)
""")

# 2.2. Izveidot tabulu ar istabām
cursor.execute("""
    CREATE TABLE IF NOT EXISTS rooms(
        room_number INTEGER PRIMARY KEY,
        room_availability BOOLEAN DEFAULT 1)
""")

cursor.execute("""
    CREATE TABLE IF NOT EXISTS staff(
        staff_id INTEGER PRIMARY KEY,
        staff_name TEXT NOT NULL,
        staff_surname TEXT NOT NULL,
        staff_job TEXT NOT NULL)
""")

# 2.3. Izveidot tabulu ar pasūtījumiem (orders)
# Fixed: Added guest_id, removed trailing comma
cursor.execute("""
    CREATE TABLE IF NOT EXISTS orders(
        order_id INTEGER PRIMARY KEY,
        guest_id INTEGER,
        room_number INTEGER,
        order_finished BOOLEAN DEFAULT 1,
        staff_id INTEGER,

        FOREIGN KEY (guest_id)
            REFERENCES guests(guest_id),
        FOREIGN KEY (room_number)
            REFERENCES rooms(room_number),
        FOREIGN KEY (staff_id)
            REFERENCES staff(staff_id)
    )
""")

# 3. Ievietot datus tabulās
cursor.execute("""
INSERT OR IGNORE INTO guests(guest_id, guest_name, guest_surname)
    VALUES(1, 'Romans', 'Zubenko')
""")

cursor.execute("""
INSERT OR IGNORE INTO rooms(room_number, room_availability)
    VALUES(1, 1)
""")

cursor.execute("""
INSERT OR IGNORE INTO staff(staff_id, staff_name, staff_surname, staff_job)
    VALUES(1, 'Antons', 'Mahno', 'Cleaner')
""")

# Fixed: Changed table name from 'pasutijumi' to 'orders' and used guest_id
cursor.execute("""
INSERT OR IGNORE INTO orders (order_id, guest_id, room_number, order_finished, staff_id)
    VALUES(1, 1, 1, 1, 1)
""")

# 4. Saņemt datus - Klienti
```

```
cursor.execute("""
    SELECT *
    FROM guests
""")

guests_list = cursor.fetchall()
print("\nKLIENTI")
for g in guests_list:
    print(g)

# 5. Saņemt datus - Pasūtījumi (Fixed joins and included staff table)
cursor.execute("""
SELECT
    guests.guest_name,
    rooms.room_number,
    staff.staff_id
    FROM orders
    JOIN guests
        ON orders.guest_id = guests.guest_id
    JOIN rooms
        ON orders.room_number = rooms.room_number
    JOIN staff
        ON orders.staff_id = staff.staff_id
""")

pasutijumi = cursor.fetchall()
print("\nPASUTIJUMI")
for p in pasutijumi:
    print(p)

# Saglabāt un aizvērt
database.commit()
database.close()
```

User prompt: rewrite this code with functions, so it works together with the previous code you wrote import sqlite3 def connect(): database = sqlite3.connect("veikals.db") database.execute("PRAGMA foreign_keys = ON") return database def add_client(name, surname): database = connect() cursor = database.cursor() cursor.execute("INSERT INTO klienti (vards, uzvards) VALUES (?, ?)", (name, surname)) database.commit() klienti_id = cursor.lastrowid database.close() return klienti_id def add_product(name, price): database = connect() cursor = database.cursor() cursor.execute("INSERT INTO preces (nosaukums, cena) VALUES (?, ?)", (name, price)) database.commit() preces_id = cursor.lastrowid database.close() return preces_id def add_order(client_id, product_id): database = connect() cursor = database.cursor() cursor.execute("INSERT INTO pasutijumi (klienti_id, preces_id) VALUES (?, ?)", (client_id, product_id)) database.commit() order_id = cursor.lastrowid database.close() return order_id def get_product(): database = connect() cursor = database.cursor() cursor.execute("SELECT preces_id, nosaukums, cena FROM preces") product = cursor.fetchall() database.close() return product def get_client(): database = connect() cursor = database.cursor() cursor.execute("SELECT klienti_id, vards, uzvards FROM klienti") client = cursor.fetchall() database.close() return client def get_orders(): database = connect() cursor = database.cursor() cursor.execute(""" SELECT pasutijumi.pasutijumi_id, klienti.vards, klienti.uzvards, preces.nosaukums, preces.cena FROM pasutijumi JOIN klienti ON pasutijumi.klienti_id = klienti.klienti_id JOIN preces ON pasutijumi.preces_id = preces.preces_id ORDER BY pasutijumi.pasutijumi_id """) client = cursor.fetchall() database.close() return client

Response: Here is how you can adapt that function-based structure to fit your **hotel database** project.

This version creates all four tables (`guests`, `rooms`, `staff`, `orders`), provides clean functions to insert and retrieve data, and includes `PRAGMA foreign_keys = ON;` to ensure foreign key enforcement works properly in SQLite.

Hotel Database with Functions

```
import sqlite3

def connect():
    database = sqlite3.connect("hotel.db")
    # Enable foreign keys support in SQLite
    database.execute("PRAGMA foreign_keys = ON")
    return database

# --- TABLE CREATION ---
def create_tables():
    database = connect()
    cursor = database.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS guests(
            guest_id INTEGER PRIMARY KEY,
            guest_name TEXT NOT NULL,
            guest_surname TEXT NOT NULL)
    """)

    cursor.execute("""
```

```
        CREATE TABLE IF NOT EXISTS rooms(
            room_number INTEGER PRIMARY KEY,
            room_availability BOOLEAN DEFAULT 1)
    """)

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS staff(
            staff_id INTEGER PRIMARY KEY,
            staff_name TEXT NOT NULL,
            staff_surname TEXT NOT NULL,
            staff_job TEXT NOT NULL)
    """)

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS orders(
            order_id INTEGER PRIMARY KEY,
            guest_id INTEGER,
            room_number INTEGER,
            order_finished BOOLEAN DEFAULT 1,
            staff_id INTEGER,

            FOREIGN KEY (guest_id) REFERENCES guests(guest_id),
            FOREIGN KEY (room_number) REFERENCES rooms(room_number),
            FOREIGN KEY (staff_id) REFERENCES staff(staff_id)
        )
    """)

    database.commit()
    database.close()

# --- ADD FUNCTIONS ---
def add_guest(name, surname):
    database = connect()
    cursor = database.cursor()
    cursor.execute("INSERT OR IGNORE INTO guests (guest_name, guest_surname) VALUES (?, ?)", (name, surname))
    database.commit()
    guest_id = cursor.lastrowid
    database.close()
    return guest_id

def add_room(room_number, availability=1):
    database = connect()
    cursor = database.cursor()
    cursor.execute("INSERT OR IGNORE INTO rooms (room_number, room_availability) VALUES (?, ?)", (room_number, avail
    database.commit()
    database.close()

def add_staff(name, surname, job):
    database = connect()
    cursor = database.cursor()
    cursor.execute("INSERT OR IGNORE INTO staff (staff_name, staff_surname, staff_job) VALUES (?, ?, ?)", (name, sur
    database.commit()
    staff_id = cursor.lastrowid
    database.close()
    return staff_id

def add_order(guest_id, room_number, order_finished, staff_id):
    database = connect()
    cursor = database.cursor()
    cursor.execute("""
        INSERT OR IGNORE INTO orders (guest_id, room_number, order_finished, staff_id)
        VALUES (?, ?, ?, ?)
    """, (guest_id, room_number, order_finished, staff_id))
    database.commit()
    order_id = cursor.lastrowid
    database.close()
    return order_id

# --- GET / SELECT FUNCTIONS ---
def get_guests():
    database = connect()
    cursor = database.cursor()
    cursor.execute("SELECT guest_id, guest_name, guest_surname FROM guests")
    guests = cursor.fetchall()
    database.close()
    return guests

def get_rooms():
    database = connect()
```

```
    cursor = database.cursor()
    cursor.execute("SELECT room_number, room_availability FROM rooms")
    rooms = cursor.fetchall()
    database.close()
    return rooms

def get_staff():
    database = connect()
    cursor = database.cursor()
    cursor.execute("SELECT staff_id, staff_name, staff_surname, staff_job FROM staff")
    staff = cursor.fetchall()
    database.close()
    return staff

def get_orders():
    database = connect()
    cursor = database.cursor()
    cursor.execute("""
        SELECT
            orders.order_id,
            guests.guest_name,
            guests.guest_surname,
            rooms.room_number,
            orders.order_finished,
            staff.staff_name,
            staff.staff_surname
        FROM orders
        JOIN guests ON orders.guest_id = guests.guest_id
        JOIN rooms ON orders.room_number = rooms.room_number
        JOIN staff ON orders.staff_id = staff.staff_id
        ORDER BY orders.order_id
    """)
    orders = cursor.fetchall()
    database.close()
    return orders

# --- TEST / EXAMPLE EXECUTION ---
if __name__ == "__main__":
    # 1. Initialize tables
    create_tables()

    # 2. Add sample data
    add_guest("Romans", "Zubenko")
    add_room(1, 1)
    add_staff("Antons", "Mahno", "Cleaner")
    add_order(guest_id=1, room_number=1, order_finished=1, staff_id=1)

    # 3. Print results to verify
    print("\nKLIENTI:")
    for g in get_guests():
        print(g)

    print("\nISTABAS:")
    for r in get_rooms():
        print(r)

    print("\nPERSONĀLS:")
    for s in get_staff():
        print(s)

    print("\nPASŪTĪJUMI:")
    for o in get_orders():
        print(o)
```

User prompt: and rewrite this code, so it can open a window and works with another files import tkinter as tk from funkcijas import (add_client, add_order, add_product) def addFromWindow(): name = name_input.get() surname = surname_input.get() add_client(name, surname) name_input.delete(0, tk.END) surname_input.delete(0, tk.END) result.config(text="klients pievienots") def addFromWindowProduct(): type = type_input.get() price = price_input.get() add_product(type, price) type_input.delete(0, tk.END) price_input.delete(0, tk.END) result2.config(text="prece pievienota") #veidojam logu window = tk.Tk() window.title("Lats datubaze") window.geometry("600x800") #virsraksts header = tk.Label(window, text = "Veikala datubaze", font = ("Arial", 20, "bold")) header.pack(pady = 20) #klienta logs client_block=tk.LabelFrame(window, text = "pievienot klientu", padx = 10, pady = 10) client_block.pack(fill = "x", padx = 20, pady = 20) tk.Label(client_block, text="vards").pack() name_input = tk.Entry(client_block, width=60) name_input.pack(pady=5) tk.Label(client_block, text="uzvards").pack() surname_input = tk.Entry(client_block, width=60) surname_input.pack(pady=5) client_button = tk.Button(client_block, text="pievienot klientu", command=addFromWindow) client_button.pack(pady=5) result = tk.Label(client_block, text="", fg="green") result.pack(pady=5) #preces logs product_block=tk.LabelFrame(window, text = "pievienot preci", padx = 10, pady = 10) product_block.pack(fill = "x", padx = 20, pady = 20) tk.Label(product_block, text="nosaukums").pack() type_input = tk.Entry(product_block, width=60) type_input.pack(pady=5) tk.Label(product_block, text="cena").pack() price_input = tk.Entry(product_block, width=60) price_input.pack(pady=5) product_button =

tk.Button(product_block, text="pievienot produktu", command=addFromWindowProduct) product_button.pack(pady=5) result2 = tk.Label(product_block, text="", fg="green") result2.pack(pady=5) #pasutijuma logs order_block=tk.LabelFrame(window, text = "pievienot pasutijumu", padx = 10, pady = 10) order_block.pack(fill = "x", padx = 20, pady = 20) tk.Label(order_block, text="klienta id").pack() client_input = tk.Entry(order_block, width=60) client_input.pack(pady=5) tk.Label(order_block, text="preces id").pack() product_input = tk.Entry(order_block, width=60) product_input.pack(pady=5) product_button = tk.Button(order_block, text="pievienot pasutijumu") product_button.pack(pady=5) window.mainloop()

Response: To make your tkinter interface work seamlessly with the hotel database functions we set up earlier, we need to adapt two things:

1. **The function imports:** Change the imports from `funkcijas` to match your hotel functions (`add_guest`, `add_room`, `add_staff`, `add_order`).
2. **The GUI blocks:** Update the fields to match the hotel tables (**Guests**, **Rooms**, **Staff**, and **Orders**) instead of the old store fields.

Assuming you save your database functions into a file named **`funkcijas.py`** (or `database.py`), here is the rewritten, fully working graphical interface code for your **Hotel Database**:

`gui.py` (or main window file)

```
import tkinter as tk
from funkcijas import (
    create_tables,
    add_guest,
    add_room,
    add_staff,
    add_order
)

# Ensure tables exist when starting the app
create_tables()

# --- BUTTON EVENT FUNCTIONS ---

def handle_add_guest():
    name = guest_name_input.get()
    surname = guest_surname_input.get()

    if name and surname:
        add_guest(name, surname)
        guest_name_input.delete(0, tk.END)
        guest_surname_input.delete(0, tk.END)
        guest_result.config(text="Klients veiksmigi pievienots!", fg="green")
    else:
        guest_result.config(text="Lūdzu aizpildiet visus laukus!", fg="red")

def handle_add_room():
    room_num = room_number_input.get()
    avail = room_avail_input.get()

    try:
        add_room(int(room_num), int(avail))
        room_number_input.delete(0, tk.END)
        room_avail_input.delete(0, tk.END)
        room_result.config(text="Istaba veiksmīgi pievienota!", fg="green")
    except ValueError:
        room_result.config(text="Kļūda! Numuriem jābūt cipariem.", fg="red")

def handle_add_staff():
    name = staff_name_input.get()
    surname = staff_surname_input.get()
    job = staff_job_input.get()

    if name and surname and job:
        add_staff(name, surname, job)
        staff_name_input.delete(0, tk.END)
        staff_surname_input.delete(0, tk.END)
        staff_job_input.delete(0, tk.END)
        staff_result.config(text="Personāls veiksmīgi pievienots!", fg="green")
    else:
        staff_result.config(text="Lūdzu aizpildiet visus laukus!", fg="red")

def handle_add_order():
    g_id = order_guest_id.get()
    r_num = order_room_num.get()
    finished = order_finished_input.get()
    s_id = order_staff_id.get()

    try:
        add_order(int(g_id), int(r_num), int(finished), int(s_id))
        order_guest_id.delete(0, tk.END)
```

```
            order_room_num.delete(0, tk.END)
            order_finished_input.delete(0, tk.END)
            order_staff_id.delete(0, tk.END)
            order_result.config(text="Pasūtījums veiksmīgi pievienots!", fg="green")
        except ValueError:
            order_result.config(text="Kļūda! ID un istabai jābūt cipariem.", fg="red")


    # --- MAIN WINDOW ---
    window = tk.Tk()
    window.title("Viesnīcas datubāze")
    window.geometry("600x900")

    # Virsraksts
    header = tk.Label(
        window,
        text="Viesnīcas Vadības Sistēma",
        font=("Arial", 18, "bold")
    )
    header.pack(pady=10)

    # --- 1. GUESTS BLOCK ---
    guest_block = tk.LabelFrame(window, text="Pievienot klientu (Guest)", padx=10, pady=5)
    guest_block.pack(fill="x", padx=20, pady=5)

    tk.Label(guest_block, text="Vārds").pack()
    guest_name_input = tk.Entry(guest_block, width=50)
    guest_name_input.pack(pady=2)

    tk.Label(guest_block, text="Uzvārds").pack()
    guest_surname_input = tk.Entry(guest_block, width=50)
    guest_surname_input.pack(pady=2)

    tk.Button(guest_block, text="Pievienot klientu", command=handle_add_guest).pack(pady=5)
    guest_result = tk.Label(guest_block, text="", fg="green")
    guest_result.pack()

    # --- 2. ROOMS BLOCK ---
    room_block = tk.LabelFrame(window, text="Pievienot istabu (Room)", padx=10, pady=5)
    room_block.pack(fill="x", padx=20, pady=5)

    tk.Label(room_block, text="Istabas numurs (Room Number)").pack()
    room_number_input = tk.Entry(room_block, width=50)
    room_number_input.pack(pady=2)

    tk.Label(room_block, text="Pieejamība (1 - brīvs, 0 - aizņemts)").pack()
    room_avail_input = tk.Entry(room_block, width=50)
    room_avail_input.insert(0, "1")  # default value
    room_avail_input.pack(pady=2)

    tk.Button(room_block, text="Pievienot istabu", command=handle_add_room).pack(pady=5)
    room_result = tk.Label(room_block, text="", fg="green")
    room_result.pack()

    # --- 3. STAFF BLOCK ---
    staff_block = tk.LabelFrame(window, text="Pievienot personālu (Staff)", padx=10, pady=5)
    staff_block.pack(fill="x", padx=20, pady=5)

    tk.Label(staff_block, text="Vārds").pack()
    staff_name_input = tk.Entry(staff_block, width=50)
    staff_name_input.pack(pady=2)

    tk.Label(staff_block, text="Uzvārds").pack()
    staff_surname_input = tk.Entry(staff_block, width=50)
    staff_surname_input.pack(pady=2)

    tk.Label(staff_block, text="Amats (Job)").pack()
    staff_job_input = tk.Entry(staff_block, width=50)
    staff_job_input.pack(pady=2)

    tk.Button(staff_block, text="Pievienot personālu", command=handle_add_staff).pack(pady=5)
    staff_result = tk.Label(staff_block, text="", fg="green")
    staff_result.pack()

    # --- 4. ORDERS BLOCK ---
    order_block = tk.LabelFrame(window, text="Pievienot pasūtījumu (Order)", padx=10, pady=5)
    order_block.pack(fill="x", padx=20, pady=5)

    tk.Label(order_block, text="Klienta ID (Guest ID)").pack()
```

```
order_guest_id = tk.Entry(order_block, width=50)
order_guest_id.pack(pady=2)

tk.Label(order_block, text="Istabas numurs (Room Number)").pack()
order_room_num = tk.Entry(order_block, width=50)
order_room_num.pack(pady=2)

tk.Label(order_block, text="Pasūtījums pabeigts? (1 - jā, 0 - nē)").pack()
order_finished_input = tk.Entry(order_block, width=50)
order_finished_input.insert(0, "1")
order_finished_input.pack(pady=2)

tk.Label(order_block, text="Darbinieka ID (Staff ID)").pack()
order_staff_id = tk.Entry(order_block, width=50)
order_staff_id.pack(pady=2)

tk.Button(order_block, text="Pievienot pasūtījumu", command=handle_add_order).pack(pady=5)
order_result = tk.Label(order_block, text="", fg="green")
order_result.pack()

# Start application
window.mainloop()
```

How to structure your project files:

Make sure both files are saved in the same folder on your computer:

1. `funkcijas.py` — Contains all the database functions (`connect`, `create_tables`, `add_guest`, `add_room`, `add_staff`, `add_order`, etc.) from the previous step.
2. `main.py` (or whatever you name this GUI script) — Contains the `tkinter` code above.

When you run `main.py`, it will import the functions from `funkcijas.py`, automatically build the database and tables if they don't exist, and open up your full window interface!

User prompt: how do i add a different sized text in a readme in guthub?

Response: To change text size in a GitHub README, you can use **Markdown headers** or **HTML tags**.

Here are the best ways to do it, from largest to smallest:

Method 1: Using Markdown Headers (Easiest)

You can use hash symbols (#) at the beginning of a line. The more hashes you use, the smaller the text becomes. There are 6 levels:

```
# Heading 1 (Largest)
## Heading 2
### Heading 3
#### Heading 4
##### Heading 5
###### Heading 6 (Smallest)
```

Method 2: Using HTML `<h>` Tags

If you want to use headers without creating a hard horizontal break line or modifying the table of contents behavior, you can use standard HTML header tags:

```
<h1>Heading 1</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>
```

Method 3: Custom Sizing with HTML `<span style="font-size: ...">` (For Exact Sizes)

If you want a specific text size that doesn't fit standard headers (like a slightly smaller paragraph or a custom title), you can use an HTML `<span>` tag with CSS styling:

```
<span style="font-size: 24px;">This text is 24px big</span>

<span style="font-size: 14px;">This text is smaller (14px)</span>

<span style="font-size: 30px;">This text is 30px big</span>
```

Method 4: Making Text Smaller or Larger with `<small>` or `<big>`

You can quickly make text one step smaller or larger using these simple HTML tags:

```
<small>This text appears smaller than normal.</small>
```

```
<big>This text appears larger than normal.</big>
```

Tip: Always check the **Preview** tab on GitHub before saving to ensure your markdown or HTML renders correctly!
